Practical Lab - Local AI with Ollama

Running and Developing Local AI Applications

Lina Taacha - FSTT

Models: qwen3.5 (9B) and gemma3 (4B) - Python 3.10 - Ollama 0.40.11 - Windows 11

1. Recap of the steps done

Step	What was done
Part 2	Checked Python 3.10 and pip, installed Ollama (version 0.40.11) and confirmed the server answers on localhost:11434.
Part 3	Downloaded qwen3.5 (6.6 GB) and ran it through the /api/generate endpoint with a helper function; tested three prompts.
Part 4	Compared a detailed prompt (role, sections, format) with a short prompt for the same topic (DAGs).
Part 5	Used ollama list, show and ps, and the same information through the REST API (/api/tags, /api/ps).
Part 6	Downloaded gemma3 (3.4 GB) and compared both models on a prime-number function.
Part 7	Built a minimal Python client (app.py, requirements.txt, README.md) and looked at Ollama's response metadata.
Parts 8-9	Built a chatbot, first stateless, then with conversation memory (history sent each time), then with /api/chat.
Part 10	Used the model as a Java code reviewer (division by zero, the age = age bug).
Part 11	Asked for JSON in the prompt, then forced it with a JSON Schema in the format field.
Part 12	Answered questions only from a supplied text document.
Part 13	Built a mini-RAG: 4 documents, a TF-IDF retriever (rag.py) and a grounded prompt.
Part 14	Vision: prepared the image analysis code (optional part).
Part 15	Built a simple Flask web page that sends a question to the model.
Part 16	Benchmarked both models (load time, latency, tokens/s, memory).
Part 17	Built the final FSTT Local AI Assistant: Flask + RAG + conversation history + JSON answers + response time.

2. Answers to the exercise questions

Exercise 4 - Running the model

Did the model produce different answers? Why?
Yes. Each prompt asked for something different (five sentences, a child, a professor), so the model adapted its vocabulary and depth. Even the same prompt can give different wording, because the model picks words with some randomness.

Is the model communicating with a cloud API?
No. The Python code calls localhost:11434, which is Ollama running on my own computer.

Where is inference being performed?
On my own machine, by Ollama, on my CPU/GPU. The model file (about 6.6 GB) was downloaded once and sits on my disk.

What happens if the Internet connection is disabled?
Nothing changes. The model is already downloaded and the request never leaves the computer. Only downloading new models needs internet.

Exercise 5 - Prompt engineering

Prompt A gave a complete, structured answer because the format was spelled out (five sections). Prompt B gave a simpler, friendlier answer because it named the audience and asked for simple language.

Element	Role in the two prompts
Role	Sets the tone. Both sounded like a teacher.
Context	Who the answer is for. B said first-year student, so the model simplified. A gave no audience.
Constraints	B asked for simple language and one example, so it stayed short. A had no limits.
Output format	A listed the sections and the model followed them. B left it open.
Specificity	The more precise the prompt, the more predictable the answer.

Exercise 6 - Model management

What do ollama list and ollama ps show?
ollama list shows the models I downloaded and their size. ollama ps shows which model is running now and how much memory it uses.

Downloaded vs loaded?
Downloaded means the model is saved on disk. Loaded means it is in memory, ready to answer.

Exercise 7 - Same prompt, different models

Model	Time (s)	Response quality /5	Code correctness /5
qwen3.5	31.3	4	4
gemma3	24.1	4	5

Faster: gemma3. **Better code:** about equal; both wrote the same correct 6k+/-1 trial-division algorithm (qwen added a useless duplicate function). **More memory:** qwen3.5. **For programming help:** gemma3, because it is faster, lighter and just as correct. **Is the largest model the best?** No, here the bigger model gave no real gain.

Part 9 - Conversation memory

Why does the model remember the name? Is the LLM itself remembering?
No, the model remembers nothing. The code stores the conversation in a Python list and sends it again with every question, so the name is simply part of the text the model receives. Without the history it would forget immediately.

Exercise 11 - Structured output

Why is structured output important?
The program needs data it can read, not free text. If the model adds extra words or changes a field name, the code breaks. With a JSON Schema, the output is always valid JSON with the right fields and types, so the app can use it directly.

Exercise 12 - Document assistant

Yes, the model followed the instruction. It answered the reinforcement learning question from the document, and for quantum computing it said "Information not available." instead of using its own knowledge.

Part 14 - Vision (conceptual)

A normal LLM only reads text; a multimodal model also accepts images. The vision model additionally receives the image (as base64), which it turns into features it can reason about. Local vision AI is useful for document analysis, OCR, industrial inspection, education and image pre-processing, without sending images to the cloud.

3. Performance benchmark (Part 16)

Model	Disk (GB)	Loaded (GB)	VRAM (GB)	Load (s)	Response (s)	Token s	Tokens/ s	Quality /5
qwen3.5	6.55	6.24	5.46	8.73	29.94	765	40.2	4
gemma3	3.35	2.88	2.88	4.32	15.76	807	86.5	4

Discussion. gemma3 was about twice as fast as qwen3.5 (86 vs 40 tokens/s) and used half the memory (2.9 vs 6.2 GB). It also loads faster. gemma3 fits entirely in the GPU, while qwen3.5 does not quite (5.5 of 6.2 GB), so part of it runs on the CPU. Quality was about the same, so the bigger model was not better here. The model should be chosen from measurements on the target machine, not from size alone.

4. Answers to the report questions (Q1-Q15)

Q1. What is Local AI?
Running an AI model on my own computer instead of using an online service.

Q2. Cloud LLM vs Local LLM?
A cloud LLM runs on a company's servers and is reached over the internet. A local LLM runs on my machine. Cloud models are usually bigger and stronger; local ones are private and free to use.

Q3. What role does Ollama play?
It downloads and runs the models and provides a local server (port 11434) that my Python code can call.

Q4. What is model quantization?
Storing the model's numbers with fewer bits (like Q4_K_M) so the file is smaller and lighter, with only a small loss in quality.

Q5. Why does model size affect memory?

Every parameter must be kept in memory while the model runs. More parameters means more memory: qwen3.5 (9B) used 6.2 GB and gemma3 (4B) used 2.9 GB.

Q6. Why can a smaller model be preferable?
It is faster, uses less memory and runs on weaker computers. For simple tasks the quality is often just as good.

Q7. Advantages of local inference?
Privacy (data stays on my machine), it works offline, there is no cost per token, and I have full control.

Q8. Disadvantages of local inference?
It needs decent hardware, can be slow, and the models are usually less capable than the biggest cloud ones. I also manage the setup myself.

Q9. Architecture of my Python application?
The browser talks to a Flask app. Flask finds relevant documents with a retriever, adds the conversation history, and sends everything to Ollama. Ollama runs the local model and returns the answer, which Flask displays with the response time.

Q10. Role of the REST API?
It lets any program talk to the model with simple HTTP requests. Python sends a POST to localhost:11434 and gets the answer back as JSON.

Q11. What is RAG?
Retrieval-Augmented Generation. The app first finds the most relevant document for the question, then gives it to the model so it answers from that text instead of guessing.

Q12. LLM vs multimodal LLM?
A normal LLM only handles text. A multimodal one can also take other inputs, such as images.

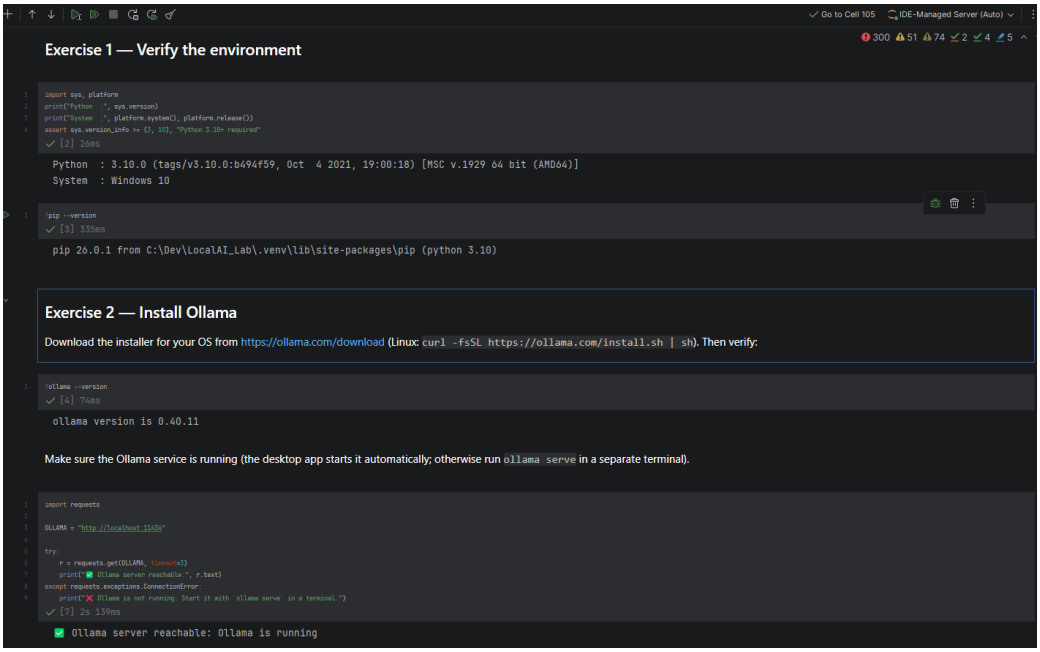
Q13. Compare two local models experimentally.
See the table in section 3: gemma3 was about twice as fast (86 vs 40 tokens/s), used half the memory (2.9 vs 6.2 GB) and had similar quality to qwen3.5.

Q14. Which model on a computer with limited RAM?
gemma3. It needs less memory, it is faster, and it still gave good answers.

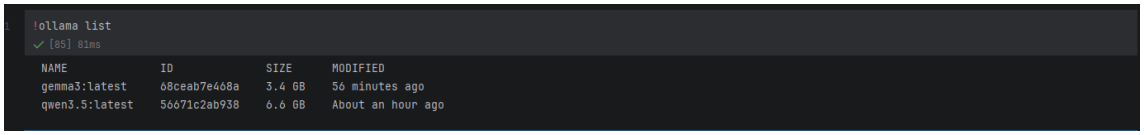
Q15. Security risks of exposing a local LLM server to a network?
Ollama has no login by default, so anyone on the network could use my computer's power, use the models and see data sent to them. Documents could also contain hidden instructions (prompt injection). I would keep it on localhost, or put it behind a firewall or a reverse proxy with authentication and HTTPS.

5. Screenshots

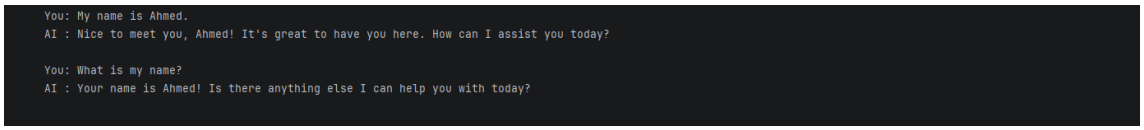
01_installation.png - Ollama installation



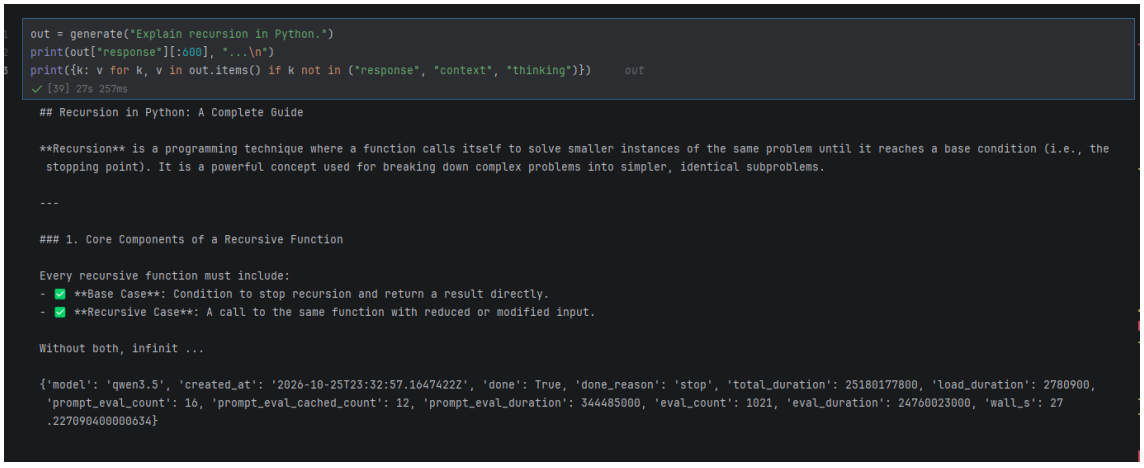
02_model.png - Model downloaded and running



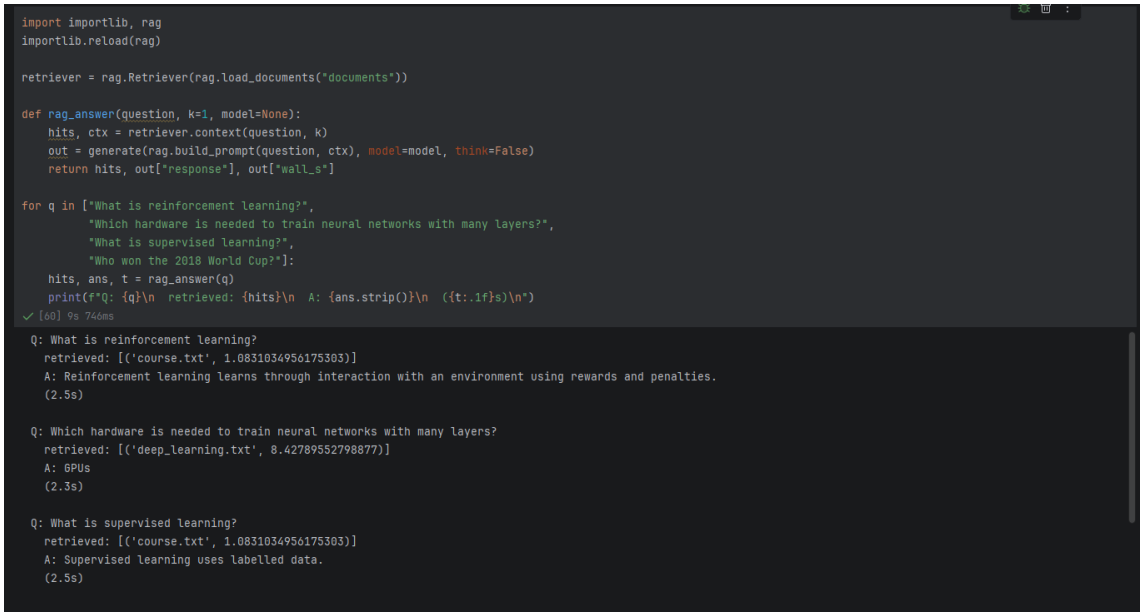
03_chatbot.png - Chatbot with memory



04_python.png - Python calling Ollama



05_rag.png - Mini-RAG



```
import importlib, rag
importlib.reload(rag)

retriever = rag.Retriever(rag.load_documents("documents"))

def rag_answer(question, k=1, model=None):
    hits, ctx = retriever.context(question, k)
    out = generate(rag.build_prompt(question, ctx), model=model, think=False)
    return hits, out["response"], out["wall_s"]

for q in ["What is reinforcement learning?",
          "Which hardware is needed to train neural networks with many layers?",
          "What is supervised learning?",
          "Who won the 2018 World Cup?"]:
    hits, ans, t = rag_answer(q)
    print(f"Q: {q}\n  retrieved: {hits}\n  A: {ans.strip()}\n  ({t:.1f}s)\n")
```

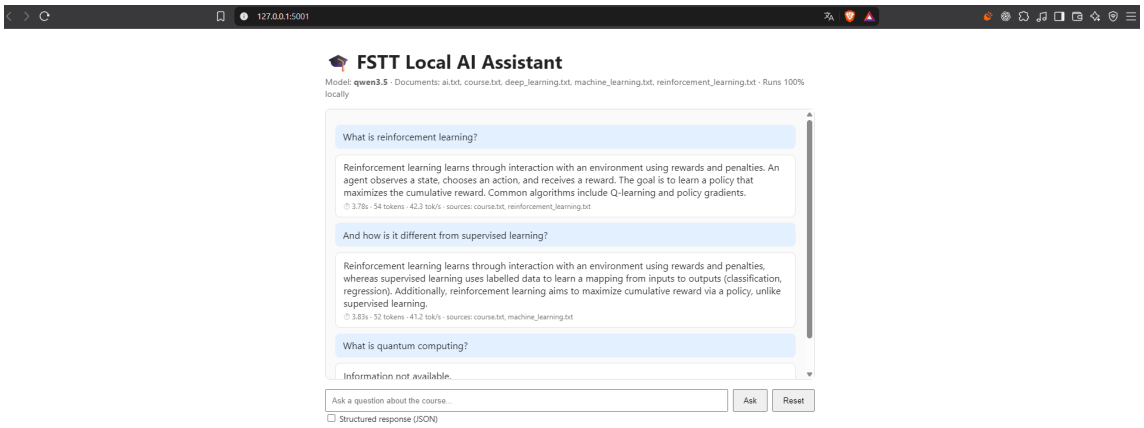
✓ [60] 9s 746ms

```
Q: What is reinforcement learning?
  retrieved: [('course.txt', 1.0831034956175303)]
  A: Reinforcement learning learns through interaction with an environment using rewards and penalties.
  (2.5s)

Q: Which hardware is needed to train neural networks with many layers?
  retrieved: [('deep_learning.txt', 8.42789552798877)]
  A: GPUs
  (2.3s)

Q: What is supervised learning?
  retrieved: [('course.txt', 1.0831034956175303)]
  A: Supervised learning uses labelled data.
  (2.5s)
```

06_webapp.png - FSTT Local AI Assistant (web app)



06_webapp_simple.png - Simple Flask web app (Part 15)

